A Serverless AWS Pipeline for Precision Agriculture Field Monitoring

Kondragunta Lakshmi Chaitanya
Student ID: X25171216
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25171216@student.ncirl.ie

***Abstract*—A field checked only on a fixed inspection schedule reveals a dry root zone, a frost event, or a fungal-risk humidity spell only after the fact. This paper presents a continuous precision-agriculture monitoring pipeline. Five field sensor types (soil moisture, temperature, humidity, light intensity, and rainfall) feed a fog node that windows and aggregates each reading stream and evaluates threshold alerts locally, forwarding only compact summaries to the cloud. A scalable serverless backend on Amazon SQS, AWS Lambda, and DynamoDB ingests those summaries and serves a live dashboard through Amazon API Gateway, with the static frontend hosted on Amazon S3. Each layer is justified against alternatives, and the system is deployed to a real AWS account rather than a local emulator. An automated suite of 39 tests passes, and end-to-end verification against the live deployment confirmed every pipeline health check reporting healthy, fresh readings arriving within seconds, and per-sensor metrics and alerts rendering correctly on the deployed dashboard. Testing against the real account additionally surfaced two defects a local emulator had masked.**

***Keywords*—precision agriculture, fog computing, Internet of Things, serverless architecture, AWS Lambda, cloud deployment, edge sensing**

I. Introduction

A field left to a fixed inspection schedule reveals a dry root zone, a frost event, or a fungal-risk humidity spell only after the fact, at the next visit. Continuous Internet of Things sensing changes that: a facilities or farm operations team can evaluate every reading against an explicit threshold rule the moment it arrives, without waiting for a scheduled walk-through. Wireless sensor deployments for precision agriculture are now well established as a way to make fields report their own condition in near real time [1].

On-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service are the five essential characteristics the National Institute of Standards and Technology uses to define cloud computing [2], and fog computing pushes that same model outward to the network edge: Bonomi et al. characterise fog computing by three traits: low latency, wide geographic distribution, and the ability to support very large numbers of nodes [3]. That definition is put into practice directly here: a fog node sitting alongside the sensors performs the windowed aggregation and threshold evaluation, and it is only the resulting aggregate-plus-alerts payload, not each raw reading, that ever crosses the network boundary into the cloud.

The design of the pipeline documented in this paper took shape around three goals. The first is a sensor and fog layer: five distinct sensor types, each with its own configurable sampling and dispatch rate, feeding a fog node that buffers, aggregates, and evaluates readings before passing them on. The second is a scalable backend, assembled from managed queue and function-as-a-service mechanisms, which turns the fog node's output into a responsive dashboard covering every sensor type. The third is proving that backend on an actual public cloud platform, not a simulation kept entirely local.

The rest of the paper is structured as follows. Section II lays out the architecture and the reasoning behind each layer's design, weighing it against alternatives that were ultimately not chosen and examining the deployment's security posture. Section III covers the implementation: the software stack, the automated tests, the CI pipeline, what the live AWS deployment actually turned up in the way of defects, and the measured evidence gathered against that deployment, from test coverage and a live load test through deployment health, cost, and limitations. Section IV closes with a critical look back at the project and where it could go next.

II. System Architecture and Design

A. Sensor and Fog Layer

The simulated deployment models five sensor types: soil moisture (sampled at two sites, field-1 and field-2), temperature, humidity, light intensity, and rainfall. Reading frequency and dispatch rate are kept as two separate, independently tunable knobs rather than one aliased value: each sensor process reads its own sampling-interval and dispatch-interval settings from environment variables, set independently of one another. Each process follows a bounded random walk around a domain-plausible value (profile-specific step size and clamp range), which keeps consecutive readings correlated instead of drawing an unrelated value every tick [4].

Each of the five profiles clamps its random walk to a physically plausible range and steps through it at a domain-appropriate rate: soil moisture walks in steps of up to 1.5 percentage points across an 8-45% range, temperature in steps of up to 0.8°C across a 2-42°C range, humidity in steps of

TABLE I
ENVIRONMENTAL ALERT RULES

Sensor Type	Rule	Alert Key
Soil moisture	avg $<$ 20	Irrigation needed
Temperature	avg $>$ 35	Heat stress
Temperature	min $<$ 3	Frost risk
Humidity	avg $>$ 90	Fungal risk
Light intensity	avg $<$ 1000	Low light
Rainfall	max $>$ 10	Heavy rain

up to 2.0 points across 25-98%, light intensity in steps of up to 6000 lux across a 0-100000 lux range, and rainfall in steps of up to 3.0 mm across 0-18 mm. The six sensor containers deployed (soil moisture at both field-1 and field-2, plus one each of the remaining four types) sample every two to four seconds and dispatch their buffered readings to the fog node every eight to sixteen seconds, with every sensor's exact interval set independently, never one shared global constant, so the sampling profile of one sensor can be tuned without affecting the others. Each dispatch reaches the fog node as a single small JSON object rather than one reading sent at a time: the sensor type, site identifier, unit, and the array of timestamp-value pairs buffered since the previous send, which for a typical four-reading dispatch serialises to roughly three hundred bytes and stays comfortably under a kilobyte even at the longest dispatch interval and highest sample rate combination.

The fog node is a Python FastAPI service that ingests batches over its /ingest endpoint, buffers them per sensor type and site under a lock, and flushes every ten seconds on a fixed timer independent of buffer size. On each flush, the fog node snapshots and clears the buffer atomically, computes each key's minimum, maximum, average, latest value, and reading count, evaluates the aggregate against the threshold rules in Table I, and dispatches every window's aggregates as one batched send to Amazon SQS, chunked at the service's ten-entry limit. This keeps the number of outbound network calls bounded regardless of how many (sensor type, site) pairs closed a window at the same moment. If that outbound send fails for any reason, the fog node logs the failure and simply moves on to the next window rather than retrying the same batch or buffering it locally for a later attempt, an accepted trade-off given how often a fresh window closes and how little a single missed one costs at this data volume.

B. Scalable Backend Layer

Every piece of the backend layer is a managed AWS primitive chosen for how it scales. An Amazon SQS standard queue, fec-agri-agg, absorbs the gap between how fast the fog node dispatches and how fast the backend can process, the same queue-based load-levelling pattern elastic cloud services generally rely on [5]. Arrivals on that queue trigger an AWS Lambda function, fec-agri-processor, through an event-source mapping, and each invocation writes one aggregated window into the fec-agri-readings DynamoDB table, keyed by sensor type as the partition key and a composite of the window-end time and the site as the sort key, the same narrow, well-known key-based lookup approach Amazon's original key-value store popularised [6] and now offers as the managed DynamoDB service [7]. That composite sort key earns its complexity: were the window-end time used alone, a soil-moisture aggregate from one field and one from another field closing in the same flush cycle would collide on one primary key, and whichever write landed second would overwrite the first silently. Appending the site identifier keeps the two fields' concurrent writes from colliding, without requiring a secondary index.

Cold-start latency is a frequent objection to Lambda-based designs: when no warm execution environment is available, an invocation pays an initialisation delay before its handler even begins running, a cost that has been measured empirically across commercial FaaS platforms [8]. Two workload-specific reasons make that cost tolerable here. The fog-to-processor path is queue-triggered and asynchronous rather than a user-facing request/response path, so a delay while a reading is durably stored never shows up as a slow page load for anyone. The dashboard also polls its Lambda frequently enough during an active demonstration that the execution environment is rarely left to go cold in the first place.

C. A FastAPI-Native Dashboard API via Mangum

The dashboard-facing Lambda, fec-agri-dashboard-api, sits behind Amazon API Gateway HTTP API and answers every /api/* route the dashboard calls: readings, summary, health, backend-stats. Rather than writing a second, parallel routing table for the Lambda deployment, this project wraps the existing FastAPI application object with the Mangum ASGI-to-Lambda adapter: each API Gateway event is translated into an ASGI call, so the same decorator-based routes that serve requests through uvicorn locally serve requests through API Gateway in production, unmodified [9]. This centralises route logic in one place instead of maintaining a hand-written dispatch table that must be kept consistent with the FastAPI version by hand. The trade-off is a heavier deployment package: bundling FastAPI, Starlette, Pydantic, and Mangum brings the Lambda's zip archive to roughly 2.9 MB, against a few kilobytes for a dependency-free handler, a cost accepted here because it removes an entire class of route-table drift bugs between the local and deployed code paths.

fec-agri-dashboard-api answers requests from four upstream sources: the DynamoDB table for readings and record counts, the SQS queue's attributes for backlog depth, fec-agri-processor's Lambda metadata to report whether the ingestion function is active, and the fog node's /health endpoint over the Elastic IP the EC2 instance is bound to. This reflects a known constraint: campus and corporate WiFi networks routinely block, by policy, the exact traffic shape of a raw EC2 public IP on a non-standard port over plain HTTP, which is why hosting the dashboard on EC2 itself would have brought back a reachability risk that the fully serverless path sidesteps entirely. The dashboard's static assets are instead served directly from an Amazon S3 bucket, and a small

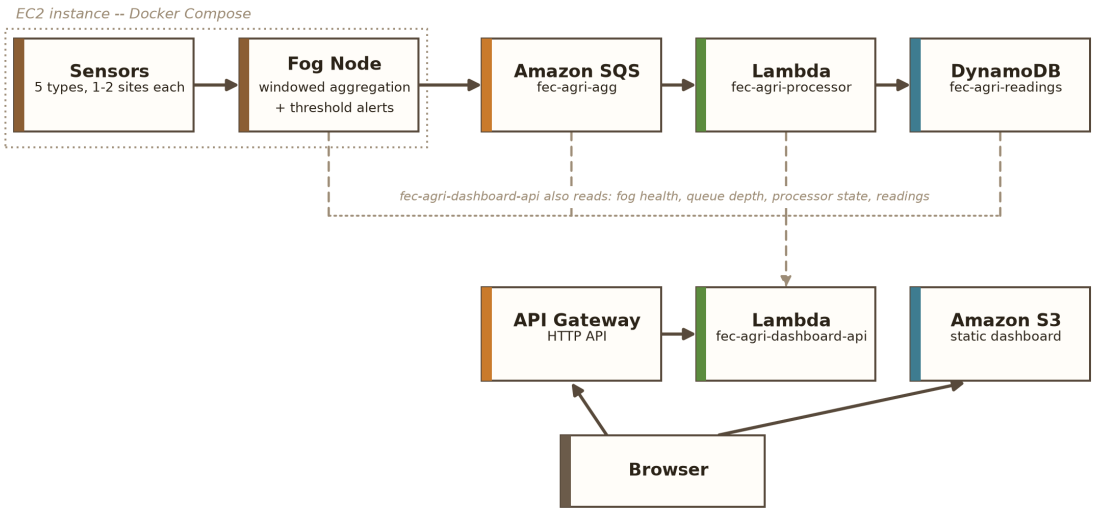

Fig. 1. Deployment topology diagram: the sensor and fog tier runs on EC2, the backend (SQS, the two Lambda functions, DynamoDB, and API Gateway) is entirely serverless, and S3 hosts the frontend independently of both.

configuration file, overwritten at deploy time with the deployed API Gateway URL, tells the browser-side JavaScript where to send its requests; it is left blank for local development, where the same FastAPI process serves both the API and the static files from one origin. On the browser side, one polling loop drives the whole page: every 2.5 seconds it fetches the summary, health, and backend-stats endpoints together, then redraws the per-sensor aggregate panels, trend charts, active alert banners, and pipeline health strip shown in Fig. 2 from the results, with a failed poll simply left for the next 2.5-second cycle to pick up rather than surfaced as an error on screen.

D. Deployment Topology

The topology this produces is drawn out in Fig. 1. Docker containers on a single EC2 instance host the sensor processes and the fog node, with Session Manager as the only way to administer that instance: no key pair exists, and port 22 has no inbound rule in its security group whatsoever. Aggregated windows flow from the fog node to SQS, SQS triggers fec-agri-processor, and fec-agri-processor writes into DynamoDB; API Gateway then fronts fec-agri-dashboard-api, which draws on that table plus the three other sources already described to answer the dashboard's REST calls, while S3 serves the dashboard's static frontend on its own, reached by the browser directly. Apart from the sensor and fog tier, every backend component in this topology sits behind a managed, serverless boundary.

E. Critical Analysis of Alternative Architectures

Each design alternative below was weighed and turned down for a concrete, statable reason. One option was collapsing ingestion and query serving into a single Lambda function instead of the two deployed here; it was rejected because the two paths are triggered differently and have different concurrency and timeout profiles, so bundling them would let a slow or failing query path starve ingestion of concurrency (or vice versa), precisely the failure mode that keeping them as separate functions is meant to prevent.

SQS was weighed against Amazon Kinesis Data Streams for the fog-to-processor link. Kinesis earns its keep when multiple independent consumers each need to replay the same ordered stream from a position of their own choosing, or when strict per-shard ordering matters in its own right, neither of which holds here. fec-agri-processor is the only thing reading the aggregated windows, and once they land in DynamoDB they are already ordered by their window-end timestamp, so Kinesis's stream-level ordering guarantee would go unused downstream. SQS's plainer consume-once-and-delete model, plus its built-in Lambda event-source-mapping integration, fits this workload with far less to operate: there is no shard count to size or reshape as throughput grows.

A ring buffer with a fixed capacity was considered here too for fog-side buffering, and set aside in favour of a plain per-window dictionary cleared on a fixed timer. A bounded ring buffer's guarantee matters most when a downstream flush could be delayed indefinitely, letting an unbounded buffer grow without limit; here the flush runs on an independent asyncio timer every ten seconds regardless of buffer size, and the sampling rate (two to four seconds per reading) means a window never accumulates more than a handful of readings per key before it is cleared. Introducing ring-buffer bookkeeping would have added complexity without a matching benefit for this specific timing profile.

A hand-written routing table for the dashboard Lambda was considered in place of the Mangum-wrapped FastAPI application actually deployed, and rejected because it would have meant maintaining two independent representations of the same five routes (one for local development, one for the Lambda deployment) with no mechanism to catch the two

drifting apart other than manual review. Reusing the existing application object was judged the safer choice for a project whose route surface changes as tests are added, at the accepted cost of a larger deployment package described in Section II-C.

DynamoDB's main rival here was Amazon RDS. A relational schema did not fit: the queries this system needs are a narrow, predictable set of key-based lookups (by sensor type, most recent first), not the ad-hoc joins across normalised tables that a relational database is built for, whereas DynamoDB's partition-key/sort-key model maps onto exactly that lookup pattern. Choosing it also meant never having to provision, patch, or pay for an always-on database instance, keeping the backend serverless end to end.

One more option, turning the fog node and sensors into scheduled Lambda functions so the entire system, not just the backend, would be serverless, was set aside rather than dismissed outright. A continuously running sensor loop does not translate cleanly onto Lambda's stateless, time-boxed invocation model without reworking the sampling logic itself, and in any case this project's cloud-deployment scope was scoped to the backend layer alone. Given the time remaining, that conversion amounted to a bigger redesign than was justified, so it is carried forward as future work in Section IV instead.

F. Security Considerations

No key pair exists for the EC2 instance at all: AWS Systems Manager Session Manager is the only way in, and with no inbound rule for port 22 in its security group, there is nothing resembling an SSH attack surface to worry about. Port 8000 is the sole inbound rule that is open, needed so fec-agri-dashboard-api can reach the fog node's health endpoint, and all that endpoint hands back is read-only, non-sensitive status information, never data access or a control-plane foothold.

Neither Lambda function gets its own IAM role; both run under the Learner Lab account's shared LabRole because the account's service-control policies block students from creating new, more tightly scoped roles. That is a genuine constraint rather than a choice: a production account would give fec-agri-processor only receive-message, delete-message, and put-item permissions, and give fec-agri-dashboard-api only read-oriented DynamoDB, SQS, and Lambda-metadata permissions, each as its own least-privilege role, and this report states that gap plainly rather than glossing over it. Authentication on the dashboard's REST API was left out deliberately, too: it surfaces nothing but aggregated, non-personal field readings, well below the bar where access control would be meaningful at this project's scale.

G. Configuration and Runtime Parameters

Every component reads its tunable behaviour from environment variables rather than from constants baked into the image, so the same container image runs unchanged in local development, LocalStack-backed integration tests, and the production AWS deployment. The fog node, the ingestion function, and the dashboard function are each configured this way, covering window length, queue and table names, the fog health URL, and the region. A single endpoint-override variable, present only in the emulator-backed configuration and deliberately absent from the AWS one, is what lets the SDK's default credential and endpoint resolution reach the real AWS account instead of the local emulator. Keeping this single variable's presence or absence as the sole switch between the two deployment targets was a deliberate simplification: it means the two configurations differ only in the handful of service definitions that genuinely differ (no emulator container, no dashboard container, an added ports mapping), not in application code that would otherwise need a conditional branch keyed on which environment is active.

III. Implementation

A. Software Components and Libraries

Python runs across the sensor, fog, processor, and dashboard-API components: FastAPI serves both HTTP-facing pieces (the fog node directly, and the dashboard API once Mangum wraps it for Lambda), and boto3 handles every AWS call. Chart.js draws the dashboard's trend visualisation client-side, and a local copy of the library ships with the page instead of a content-delivery-network reference, leaving no third-party runtime dependency behind. Day-to-day development and CI, meanwhile, run Docker Compose against LocalStack, an open-source AWS emulator, which lets the same SQS, DynamoDB, and Lambda calls used in production be exercised without ever touching the real AWS account. All four AWS-facing components pin boto3 to the same release, 1.35.90, so one library version governs the client behaviour exercised in unit tests, LocalStack-backed integration tests, and the live Lambda runtime, eliminating the risk of three versions silently drifting apart.

The window-reduction function implements the standard windowed minimum, maximum, average, latest, and count aggregation described in Section II. Every other file in this project (the sensor loop, the alert-rule table, the SQS publisher, and the dashboard routes) is an independent implementation written for this domain.

B. Testing Strategy

Each module carries its own pytest suite, run against hand-written fake AWS clients and the web framework's own test client rather than through any mocking library, which means every assertion checks the literal request shape or response body a call actually produces, with nothing standing between the test and the code. The sensor tests cover the bounded random-walk generator staying within its profile's clamp range across hundreds of iterations. The fog-layer tests cover the aggregation arithmetic, four of the six alert rules (irrigation, the two temperature rules, and heavy rain) using values comfortably past their threshold and leaving the exact boundary itself untested, and the /ingest endpoint's buffering behaviour end to end; humidity's fungal-risk and light intensity's low-light rules have no test coverage at all. This section states that gap plainly; it does not gloss over it. The publisher tests cover the batching boundary explicitly: a 23-message window

is asserted to produce batches of ten, ten, and three, not one call per message. The dashboard tests cover the DynamoDB pagination fix directly, with a three-page fake table (500, 500, and 214 items) asserting the summed count is exactly 1214, and a separate fake that raises on scan, asserting the endpoint degrades to a null count instead of a 500 error.

C. Continuous Integration and Deployment

A GitHub Actions workflow runs on every push and pull request, organised as two dependent jobs, not one. The first, unit-tests, installs the project's Python 3.12 dependencies and runs the full pytest suite in isolation, with no Docker containers and no network calls at all. The second, integration, only starts once unit-tests has passed, and brings up the full LocalStack-backed stack before running an end-to-end pipeline check against it, tearing the stack down afterwards regardless of the outcome and capturing container logs specifically when it fails. Gating the slower, container-based integration job behind the fast unit-test job means a trivial syntax or logic error is caught in seconds, not after several minutes of container startup. Separately from CI, a standalone load-testing script fires a configurable burst of synthetic messages straight onto the live queue, deliberately tagged with sensor-type names outside the real five so they can never land in the partitions the live dashboard reads; this is the same script behind the live load-test evidence presented in Section III-F.

D. Real AWS Deployment and Defects Discovered

Every piece (sensor and fog tier, SQS queue, both Lambda functions, the DynamoDB table, the API Gateway endpoint, and the S3-hosted frontend) went live on an actual AWS Academy Learner Lab account in us-east-1, giving the backend a real public-cloud test rather than only a local emulator's approximation. Two defects came out of that deployment that the LocalStack-backed test suite, run locally, had let through.

First, a single COUNT-select Scan call was how fec-agri-dashboard-api's backend-stats endpoint totalled the items stored in DynamoDB. A COUNT-select scan, however, only counts whatever page it happens to read, capped at roughly 1 MB of table data, so anything larger returns a pagination cursor that has to be chased down with further scans, or the count quietly falls short. This project's own data volume never grew large enough to expose the bug visibly, yet it was caught and fixed anyway, since a COUNT-select Scan's single-page limit is a well-known DynamoDB pagination pitfall regardless of workload. The fix is a generator that follows the pagination cursor across pages and sums every page's count, a straightforward pagination-following implementation rather than a hand-rolled loop.

Second, the fog publisher originally sent one SQS message per aggregate in a loop: functionally correct but wasteful, issuing one outbound API call per aggregate even when several (sensor type, site) windows closed in the same flush cycle. The fix batches messages at the ten-entry send limit, and the fog node's flush loop now calls it once per window instead of looping the earlier single-message send, cutting the number of outbound SQS calls and the per-message overhead that comes with each one.

TABLE II
AUTOMATED TEST SUITE BY MODULE

Module	Tests	Focus
sensors	4	bounded random walk, profile coverage
fog (ingest+aggregation+alerts)	11	buffering, windowed aggregation, threshold rules
fog (publisher)	6	SQS batching, ten-entry chunk boundary
backend/processor	6	DynamoDB item shape, sort-key logic
backend/dashboard	12	routes, pagination fix, health/backend-stats

Trusting the unit test suite alone was not enough for the pagination fix: it was re-verified against the live deployment too, and the items-in-table count reported in Section III-G checks out correctly against the deployed table. The batching fix is verified by the publisher test suite's assertion that a 23-message window produces batches of ten, ten, and three, not by the live burst load test in Section III-F, which deliberately bypasses the fog node's publish path; it exercises the queue directly with individual single-message send calls to gather queue-depth evidence, and was never intended to exercise the batch API. Anyone wanting to inspect this directly can find it at [GitHub repository URL], where the source code lives with both fixes included.

E. Automated Test Coverage

The state of the automated test suite at the final verified deployment is laid out in Table II: all 39 tests pass in the LocalStack-backed development environment, and, everywhere the suite touches real AWS client-construction logic, they pass just as cleanly against the fixes verified in the deployed AWS environment from Section III-D.

F. Live Load Test

300 synthetic messages, tagged with sensor-type names kept deliberately outside the pipeline's real five so they could never touch the partitions the live dashboard reads, landed on the deployed fec-agri-agg queue in a 4.49-second burst, about 67 messages per second. Right after that burst finished, Amazon SQS's approximate message-count attributes both reported a queue depth of zero, a reading that on its face looks like a failed send. Querying DynamoDB directly across the five synthetic partitions told a different story: all 300 messages, sixty to a partition, had already been received, processed, and written by fec-agri-processor before the follow-up check even ran.

SQS's own documentation explains this outcome, not a broken pipeline: its approximate-count attributes are eventually consistent by design, so under-reporting immediately after a burst is expected behaviour, not a symptom of something failing. LocalStack's emulation is more immediately consistent than the real service, so it would never have surfaced this; only testing against the actual deployed queue could.

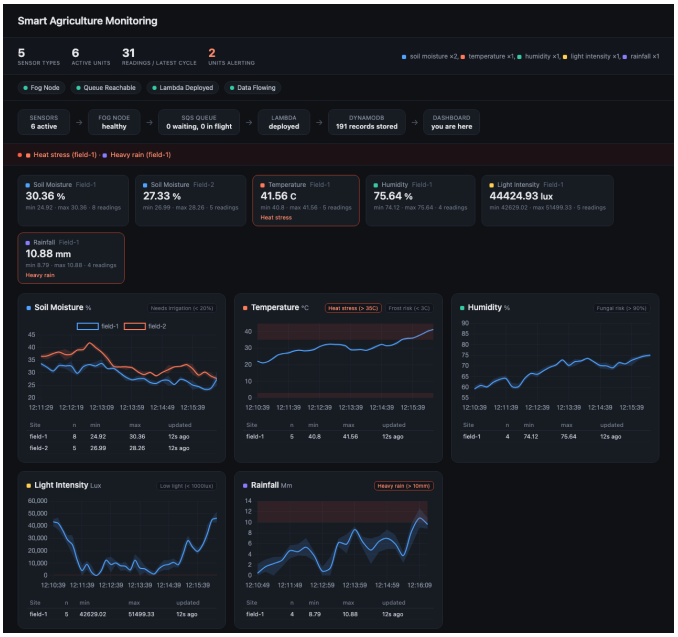

Fig. 2. The S3-hosted Smart Agriculture Monitoring dashboard during live verification, showing per-sensor aggregate panels, trend charts, active alert banners, and the end-to-end pipeline health strip.

G. Live Deployment Health

Two calls to the deployed API's /api/health endpoint, fifteen seconds apart, came back with freshest-age readings of 7.46 and then 1.88 seconds, and items-in-table counts that behaved like a live, continually updating pipeline rather than static or cached data. Every underlying AWS resource (the DynamoDB table, the SQS queue, both Lambda functions, the API Gateway API, and the EC2 instance) was checked independently through direct AWS CLI calls rather than taken on the application's own self-reported status: the DynamoDB table reported status ACTIVE under on-demand billing, the SQS queue's ARN resolved correctly, both Lambda functions reported state Active with their last update Successful, the event-source mapping between the queue and fec-agri-processor reported state Enabled, and the EC2 instance reported state running against its bound Elastic IP. Fig. 2 shows the S3-hosted dashboard rendering that same live state in the browser during this verification session.

H. Cost Considerations

At this project's data volume, only the EC2 instance falls outside AWS's request-based free tier. Everything else stays within it: DynamoDB's on-demand billing charges per request rather than provisioned throughput, both Lambdas' invocation counts sit comfortably under the perpetual 1-million-request Lambda allowance, and the API Gateway HTTP API and SQS queue are billed the same per-request way, with nothing charged while idle. EC2 is the one piece billed continuously, since it keeps the fog node and sensor containers alive as long-running processes rather than on-demand invocations; it can also be stopped between demo sessions with no effect on the dashboard or its API, whose only dependency on it is the fog-health field of /api/health and the arrival of fresh sensor data. This asymmetry, one continuously billed component against an otherwise fully request-billed backend, is itself evidence for the critical analysis in Section II-E: converting the sensor and fog tier to a scheduled-Lambda model would remove the architecture's one remaining continuously running, continuously billed component.

I. Limitations

This evaluation carries three limitations worth naming rather than leaving unstated. The fog and sensor tier is still a single EC2 instance and, unlike the fully redundant AWS-managed backend behind it, remains a single point of failure for that tier alone; Section II-E explains why moving it to a fully serverless model was deferred rather than attempted, but that leaves a real, unresolved availability gap rather than a theoretical one. The Learner Lab account backing this deployment is also session-based rather than persistent, so the kind of sustained, multi-day uptime check a production system would warrant simply was not achievable here; what this section reports is verified behaviour within one continuous session, not evidence of long-run reliability. And both Lambda functions share a single broadly scoped IAM role rather than two least-privilege ones, a constraint imposed by the Learner Lab account rather than a security choice, as Section II-F discusses.

IV. Conclusion

The goal driving this project was a complete fog and edge computing pipeline for precision agriculture field monitoring: a configurable sensor and fog layer, a scalable cloud backend split across two independently deployed Lambda functions, and deployment and testing on a genuine public cloud platform, paired with a critical weighing of each layer's architectural decisions against the alternatives on offer. That goal was met: all three layers were built, tested, and shipped to a production AWS account, not left as a local simulation.

The DynamoDB pagination defect is a concrete example of why testing against a real deployment mattered here: it cleared the full local test suite and a complete LocalStack-backed integration run without a single failure, then surfaced only once the real AWS deployment exercised it, so a build that never moved past the emulated stage would have carried that defect forward without anyone noticing. The live load test adds a second, independent example: SQS's approximate-count consistency model is a platform behaviour no local test can exercise, however cleanly that test passes, since the underlying emulator was never built to reproduce it.

Writing the application code turned out to be the easy part; what actually proved difficult was uncovering the platform and IAM constraints built into the AWS Academy Learner Lab account itself (the shared LabRole, the session-based account lifetime), constraints that shaped real design decisions rather than sitting on the sidelines. Should those account restrictions ever lift, worthwhile future work includes moving the fog and sensor tier off a continuously running EC2 instance and onto a

fully event-driven, scheduled-Lambda model, and splitting the single shared LabRole this deployment had to live with into two separate least-privilege roles, one for fec-agri-processor and one for fec-agri-dashboard-api.

REFERENCES

[1] M. Ayaz, M. Ammad-Uddin, Z. Sharif, A. Mansour, and E. H. M. Aggoune, "Internet-of-Things (IoT)-Based Smart Agriculture: Toward Making the Fields Talk," *IEEE Access*, vol. 7, pp. 129 551–129 583, 2019.

[2] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, Special Publication 800-145, Sep. 2011.

[3] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st ACM Mobile Cloud Computing Workshop (MCC '12)*, Helsinki, Finland, Aug. 2012, pp. 13–16.

[4] T. Ojha, S. Misra, and N. S. Raghuwanshi, "Wireless Sensor Networks for Agriculture: The State-of-the-Art in Practice and Future Challenges," *Computers and Electronics in Agriculture*, vol. 118, pp. 66–84, 2015.

[5] N.-L. Tran, S. Skhiri, and E. Zimányi, "EQS: An Elastic and Scalable Message Queue for the Cloud," in *Proc. 2011 IEEE 3rd Int. Conf. Cloud Computing Technology and Science (CloudCom)*, Athens, Greece, 2011, pp. 391–398.

[6] G. DeCandia *et al.*, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, Stevenson, WA, USA, Oct. 2007, pp. 205–220.

[7] M. Elhemali *et al.*, "Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service," in *Proc. 2022 USENIX Annu. Tech. Conf. (USENIX ATC '22)*, Carlsbad, CA, USA, Jul. 2022, pp. 1037–1048.

[8] M. Shahrad, J. Balkind, and D. Wentzlaff, "Architectural implications of function-as-a-service computing," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO-52)*, Columbus, OH, USA, Oct. 2019, pp. 1063–1075.

[9] A. Akbulut and H. G. Perros, "Software Versioning with Microservices through the API Gateway Design Pattern," in *Proc. 2019 9th Int. Conf. Advanced Computer Information Technologies (ACIT)*, 2019, pp. 289–292.